

VaultBox — Stage 0 Reference

Bare Single-Request Upload

A complete reference for Stage 0. Every section references real file paths and function names from the codebase. Read this before touching Stage 1.

0. Tech Stack — What Each Piece Is, and How Stage 0 Uses It

Every technology below earns its place — nothing here is decorative. This section explains what each one is in general, then exactly how Stage 0's code applies it.

Node.js

What it is: a JavaScript runtime built on Chrome's V8 engine that runs JS outside the browser, with an event-loop and non-blocking I/O model — a single thread handles many concurrent operations by never waiting idle on I/O (disk, network) before moving to the next task.

How Stage 0 uses it: every stream in the upload/download pipeline (`req`, `busboy`, `fileStream`, `sizeCounter`, the MinIO SDK's HTTP client) relies on Node's non-blocking I/O model. This is *why* the streaming pipeline in Section 4 works without threads or manual buffering — Node's event loop moves each chunk along as it arrives, rather than blocking the whole process while one file uploads.

TypeScript

What it is: a typed superset of JavaScript that compiles down to plain JS. It catches type errors (wrong argument types, missing fields, typos on object properties) at compile time instead of at runtime.

How Stage 0 uses it: every file (`server.ts`, `routes/files.ts`, `lib/minio.ts`, `lib/prisma.ts`) is `.ts`. Prisma auto-generates TypeScript types from `schema.prisma`, so `prisma.file.create({...})` gets compile-time checking that you're passing a `filename: string`, `size: number`, etc. — a typo like `flename` fails the build instead of failing silently at 2am in production.

Express

What it is: a minimal HTTP server framework for Node.js. It handles routing (matching a URL + method to a handler function), middleware chaining (functions that run before your

handler — auth checks, body parsing, logging), and response helpers (`res.json()`, `res.status()`).

How Stage 0 uses it: `server.ts` creates the `app`, mounts `filesRouter`, and calls `app.listen()`. `routes/files.ts` defines the three endpoints as Express route handlers (`router.post('/upload', ...)`, `router.get('/files/:id', ...)`, `router.get('/files', ...)`). Express is deliberately *not* used for multipart body parsing here — that's busboy's job — Express just routes the request to the handler and lets that handler take over the raw request stream.

busboy

What it is: a low-level, streaming multipart/form-data parser for Node.js. Unlike higher-level libraries (e.g. `multer`), busboy never buffers the uploaded file to disk or memory by default — it emits events (`'file'`, `'field'`, `'finish'`, `'error'`) as it parses the incoming stream, and hands you a raw `Readable` stream for each file part.

How Stage 0 uses it: in `POST /upload`, `busboy({ headers: req.headers })` reads `Content-Type` to find the multipart boundary, then `req.pipe(bb)` starts feeding it bytes. Its `'file'` event callback is where `storageKey` is generated and the `fileStream` it hands back becomes the start of the streaming pipeline described in Section 4. Busboy was chosen specifically *because* it doesn't buffer — that's the precondition for the whole "never sits fully in memory" design decision.

MinIO SDK (`minio` npm package)

What it is: a JavaScript client library for the S3 HTTP API. Despite the name, it isn't MinIO-specific — it's a general-purpose S3-protocol client that happens to be maintained by the MinIO project. It exposes methods like `putObject`, `getObject`, `bucketExists`, `makeBucket` that internally format S3-compliant HTTP requests (signed with S3's auth scheme) and parse the XML responses.

How Stage 0 uses it: `lib/minio.ts` instantiates one `Client` singleton, configured with `pathStyle: true` and pointed at moto's port `4566`. `routes/files.ts` calls `minioClient.putObject(...)` on upload and `minioClient.getObject(...)` on download. Because this SDK only speaks the standardized S3 wire protocol (see Section 6), it works unmodified against moto, real MinIO, or real AWS S3 — the client has no idea which one it's talking to.

moto server (`motoserver/moto`)

What it is: an open-source AWS service emulator, originally built as a Python testing library (`moto` mocks `boto3` calls in unit tests) and also shippable as a standalone server (`motoserver/moto` Docker image) that exposes real HTTP endpoints mimicking AWS APIs — S3 included. It implements the same REST/XML wire protocol AWS itself uses, so any S3-compatible client can talk to it without knowing it isn't real AWS.

How Stage 0 uses it: it's the `s3mock` service in `docker-compose.yml`, mapped to host port `4566`. It replaced MinIO after MinIO became unreachable (see Section 6) — moto is the object store Stage 0 actually writes bytes to and reads bytes from, standing in for what would be a real S3 bucket in production. It's genuinely doing storage work here, not just returning canned responses — files really persist inside its container for the container's lifetime.

PostgreSQL

What it is: an open-source relational database. Data lives in tables with typed columns and enforced constraints (primary keys, foreign keys, uniqueness); you query it with SQL, and it supports transactions (multiple writes that either all succeed or all roll back together).

How Stage 0 uses it: the single source of truth for file *metadata* — never file bytes. The `files` table holds one row per uploaded file. `GET /files` relies on Postgres's `ORDER BY` and `WHERE` to do the filtering and sorting that an object store like moto fundamentally cannot do on its own (S3-style stores have no query language).

Prisma ORM

What it is: an ORM (object-relational mapper) and query builder for TypeScript/Node.js. You declare your schema once in `schema.prisma`; Prisma generates a fully-typed client (`PrismaClient`) and SQL migration files from it, so your application code writes `prisma.file.create({...})` instead of hand-written SQL strings, with compile-time type safety on every field.

How Stage 0 uses it: `prisma/schema.prisma` declares the `File` model; `prisma migrate` generated the SQL migration under `prisma/migrations/20260917032034_stage0_bare_upload/`. `lib/prisma.ts` exports the singleton client instance (with the `globalThis` hot-reload fix described in Section 2). `routes/files.ts` uses it for every database operation — `create`, `findUnique`, `findMany` — none of it hand-written SQL.

Redis

What it is: an in-memory key-value data store, typically used as a cache or for fast, ephemeral state (counters, session data, queue coordination) because reads/writes are extremely fast — data lives in RAM, not on disk, by default.

How Stage 0 uses it: nothing yet — it's provisioned in `docker-compose.yml` (`redis:7-alpine`, host port `6380`) but no application code touches it. It's present now purely so Stage 1's chunk-state tracking can start using it immediately without a docker-compose change mid-stage.

Docker & Docker Compose

What it is: Docker packages an application and its dependencies into an isolated, reproducible container. Docker Compose defines *multiple* containers (services) that need

to run together — with their networking, ports, volumes, and startup health checks — in one declarative YAML file, started together with one command.

How Stage 0 uses it: `docker-compose.yml` defines the three infrastructure services (`postgres`, `s3mock`, `redis`) with health checks and offset host ports so they don't collide with the SimpleCI project on the same machine. `docker compose up` starts all three with one command — the Express API itself runs outside Docker (via `npm run dev`), connecting to these three containers over `localhost` at their mapped ports.

uuid (`uuidv4`)

What it is: a library for generating universally unique identifiers — random 128-bit values with a vanishingly small collision probability, used as identifiers where you can't or don't want to rely on an auto-incrementing counter.

How Stage 0 uses it: two places. `id: String @id @default(uuid())` in the Prisma schema lets Postgres generate the primary key for each `File` row. In `routes/files.ts`, `uuidv4()` generates the unique path segment inside `storageKey` (`files/<uuid>/<filename>`) specifically so two different users uploading a file with the same name — `photo.jpg` — never collide in the object store.

dotenv

What it is: a small library that reads a `.env` file and loads its key-value pairs into `process.env`, so secrets and environment-specific config (database URLs, ports, credentials) stay out of source code and out of version control.

How Stage 0 uses it: `import 'dotenv/config'` as the very first line executed in `server.ts` — loading `.env` before any other module (which may read `process.env` at import time, like `lib/minio.ts` and `lib/prisma.ts`) gets evaluated. This is why import order matters here specifically.

ts-node-dev

What it is: a development tool that runs TypeScript directly (no separate compile step) and automatically restarts the process when a source file changes — a hot-reload loop for local development.

How Stage 0 uses it: it's what `npm run dev` invokes to run `server.ts`. It's also the direct cause of the `globalThis` singleton pattern in `lib/prisma.ts` (Section 2) — its hot-reload behavior re-evaluates modules on every save, which would otherwise spawn a fresh `PrismaClient`/connection pool each time.

1. Where This Stage Fits in the Whole Project

VaultBox separates every file operation into two concerns, stored in two separate systems.

Bytes — the raw file data — go to an S3-compatible object store. In development that's `motoserver/moto` running in Docker; in production it would be a real AWS S3 bucket or any S3-compatible service. Object stores are built for arbitrary-size binary blobs with parallel chunked I/O and cheap egress. They don't do queries.

Facts about bytes — filename, who owns it, how large it is, when it was uploaded — go to PostgreSQL via Prisma. Postgres is where you do relational queries: list all files for a user, sum storage by owner, filter by date, join to a dedup hash table.

Express is the traffic cop between the two. It never holds onto file bytes. It decides where bytes go and records that decision in Postgres.

Stage 0 proves the minimum: Express can receive a multipart file, stream it to the object store without buffering the whole thing in RAM, and record the metadata in Postgres. The download path reverses that — read the metadata, stream bytes back from the store to the HTTP client.

Everything later layers on top of this foundation:

Stage	Adds
1	Client splits the file into 5MB chunks, uploads 4 in parallel
2	Client resumes after a dropped connection; SHA-256 checksum per chunk
3	Deduplication — same bytes from two users stored once
4	Pre-signed URLs — client uploads directly to S3, bypassing Express
5	Versioning — old file copies kept on overwrite
6	Sharing and per-user storage quotas
7	Cleanup jobs — background garbage collection of orphaned chunks

Stage 0 does none of that. One request, one file, one round-trip.

2. File-by-File Walkthrough

`apps/api/src/server.ts`

Single responsibility: boot the application in the correct order and expose the HTTP server.

Nothing calls this file — it's the entrypoint. `ts-node-dev` executes it directly when you run `npm run dev`.

Boot sequence, in exact order:

1. `import 'dotenv/config'` fires first as a side effect, loading `.env` into `process.env` before any other code runs. Must be the first import.
2. `express()` creates the app instance.
3. `app.use(express.json())` registers JSON body parsing. Multipart uploads go through busboy, not this — but future endpoints that accept JSON bodies will need it.
4. `app.get('/health', ...)` registers a cheap liveness probe.
5. `app.use('/', filesRouter)` mounts all three file endpoints from `routes/files.ts`.
6. `app.use(404 catch-all)` catches any unmatched route.
7. `start()` runs — the critical async function.

Inside `start()`:

- `await prisma.$connect()` opens the Postgres connection pool. Throws immediately if Postgres is unreachable.
- `await ensureBucket()` from `lib/minio.ts` creates the S3 bucket if it doesn't exist. Throws immediately if moto is unreachable.
- `app.listen(PORT, ...)` is only reached after both checks pass.

Design decision: the simpler approach is to call `app.listen` immediately and let the first request discover a missing bucket. The fail-fast pattern here gives a clear error and `process.exit(1)` instead of a mystery 500 on the first upload — you know immediately on startup if infrastructure is down, not after the first customer hits the endpoint.

`apps/api/src/routes/files.ts`

Single responsibility: define the three HTTP endpoints and coordinate busboy, the MinIO SDK, and Prisma for each one.

`server.ts` mounts this via `app.use('/', filesRouter)`.

POST /upload

1. Read `req.headers['x-user-id']` and trim it. Reply `400` if missing.

2. Create a busboy instance: `const bb = busboy({ headers: req.headers })`. Busboy reads `Content-Type` to extract the multipart boundary string.
3. Register `bb.on('file', (_fieldname, fileStream, info))` — fires when busboy finds a file field.
4. Inside the file callback:
 - Generate `storageKey = 'files/' + uuidv4() + '/' + filename` — the UUID prefix prevents collisions if two users upload `photo.jpg`.
 - Create a `Transform` stream called `sizeCounter` that increments `size` by `chunk.length` on each chunk and calls `cb(null, chunk)` to pass it through unchanged.
 - Pipe `fileStream` into `sizeCounter` to get `countedStream`.
 - Call `minioClient.putObject(BUCKET_NAME, storageKey, countedStream, undefined, { 'Content-Type': mimeType })` — opens an HTTP PUT to moto; bytes start flowing.
 - In the `.then()` after moto confirms the write: call `prisma.file.create({ filename, size, owner, storageKey, mimeType })`.
 - In the next `.then()`: call `reply(201, { id, filename, size, owner, createdAt })`.
5. Register `bb.on('finish')` — fires after all multipart parts are parsed. If no file field was seen and no reply sent, reply `400`.
6. Register `bb.on('error')` — busboy parse failure, reply `500`.
7. `req.pipe(bb)` — the line that starts the entire pipeline. Nothing moves until here.

Design decision — the reply guard: the handler defines a local `reply` function that checks a `responded` boolean before calling `res.status().json()`. Without it, busboy's `'finish'` event fires *after* `'file'` completes — a successful upload would call `reply(201)`, then `'finish'` would immediately call `reply(400, 'no file found')`, attempting to set headers on an already-sent response, which Node.js treats as an error.

`GET /files/:id`

1. `await prisma.file.findUnique({ where: { id: req.params.id } })`. Reply `404` if null.
2. `await minioClient.getObject(BUCKET_NAME, file.storageKey)` — returns a Node.js `Readable` stream pointing at the object in moto.
3. Set `Content-Type` from `file.mimeType` (fallback `application/octet-stream`), `Content-Disposition` with the original filename, `Content-Length` from `file.size`.
4. `stream.pipe(res)` — bytes flow from moto directly into the HTTP response.
5. `stream.on('error', ...)` calls `res.destroy()` — if moto dies mid-stream, destroy the socket rather than leave the client hanging with an incomplete download.

Design decision: `Content-Length` is served from `file.size` in Postgres, not a `headObject` call to moto. The byte count was computed exactly during upload via `sizeCounter`, so it's already in Postgres — using it here saves a round-trip on every download.

`GET /files`

1. Read and validate `X-User-Id` header. Reply `400` if missing.
2. `await prisma.file.findMany({ where: { owner }, orderBy: { createdAt: 'desc' }, select: { id, filename, size, mimeType, createdAt } })`.

3. `res.json({ files })`.

Design decision: `storageKey` is deliberately excluded from `select`. It's an internal S3 path — clients have no business constructing storage paths from API responses.

`apps/api/src/lib/minio.ts`

Single responsibility: configure the MinIO SDK client singleton and export `ensureBucket`.

`server.ts` imports `ensureBucket` and calls it at startup. `routes/files.ts` imports `minioClient` and `BUCKET_NAME` for every endpoint.

```
□
typescript
endPoint: process.env.S3_ENDPOINT ?? 'localhost'
port:    parseInt(process.env.S3_PORT ?? '4566', 10)
useSSL:  false
accessKey: process.env.S3_ACCESS_KEY ?? 'test'
secretKey: process.env.S3_SECRET_KEY ?? 'test'
pathStyle: true
```

All values come from `.env` with hardcoded fallbacks so the server still starts if you forget to copy `.env.example`. The `pathStyle` flag is explained fully in Section 6.

`ensureBucket` calls `minioClient.bucketExists(BUCKET_NAME)`. If the bucket exists (server restart without restarting moto), logs "ready" and returns. If not, calls

`minioClient.makeBucket(BUCKET_NAME, process.env.S3_REGION ?? 'us-east-1')`. Called exactly once in `start()`, before `app.listen`.

`apps/api/src/lib/prisma.ts`

Single responsibility: export exactly one `PrismaClient` instance for the entire process lifetime.

Both `server.ts` (for `$connect`) and `routes/files.ts` (for all queries) import from here.

```
□
typescript
```

```
const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma =
  globalForPrisma.prisma ?? new PrismaClient({ log: ['query', 'error', 'warn'] });

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma;
}
```

`ts-node-dev` hot-reloads modules on every file save. Without this pattern, each save creates a new `PrismaClient` and a new connection pool — after roughly 10 saves you exhaust Postgres's `max_connections`. The `globalThis` cache survives module re-evaluation so you always get one pool regardless of how many hot-reloads happen.

In production, `NODE_ENV` is `'production'`, the cache is skipped, and `new PrismaClient()` runs exactly once at cold start.

`log: ['query', 'error', 'warn']` in development prints every SQL statement Prisma executes to the terminal, so you can see exactly what queries your endpoints fire without opening a separate DB client.

`apps/api/prisma/schema.prisma`

Single responsibility: declare the Postgres schema Prisma uses to generate migration SQL and the TypeScript client.

The `datasource` block points at `DATABASE_URL` from `.env` — in development, `postgresql://vaultbox:vaultbox_secret@localhost:5433/vaultbox_db`, matching the `postgres` service in `docker-compose.yml` exactly.

The `File` model maps to a Postgres table called `files` (lowercase plural, set by `@@map("files")`). Without `@@map`, Prisma would name the table `"File"` — which works but violates standard SQL naming conventions. The model name stays `File` in TypeScript (PascalCase singular); the table name is `files` in Postgres (lowercase plural). `@@map` reconciles the two.

Every field is covered in Section 5.

`docker-compose.yml`

Single responsibility: declare the three local development infrastructure services — Postgres, moto, and Redis.

Service	Image	Host port	Container port
postgres	postgres:16-alpine	5433	5432
s3mock	motoser/moto:latest	4566	5000
redis	redis:7-alpine	6380	6379

All ports are offset from their standard values. Postgres normally runs on `5432`, Redis on `6379` — another project on the same machine (SimpleCI) owns those ports. On standard ports, `docker compose up` would fail with a bind error.

Redis is unused in Stage 0. It's in the Compose file because Stage 1 needs it to track chunk state — which chunks are received, which are missing. Adding it now means Stage 1 can start using it without modifying infrastructure files.

All three services have healthchecks. Docker marks a service `(healthy)` vs `(starting)` based on these. The Express server's `start()` function acts as its own dependency gate — it doesn't use `depends_on`; it just calls `prisma.$connect()` and `ensureBucket()` and lets those throw if dependencies aren't ready.

3. Full Request Trace

POST /upload



```
curl.exe -s -X POST http://localhost:4000/upload -H "X-User-Id: user_1" -F "file=@test.txt"
```

1. curl sends an HTTP POST with `Content-Type: multipart/form-data; boundary=----` `FormBoundaryXXX` and the file bytes as the body.
2. TCP bytes arrive at the Express server on port `4000`.
3. Express matches the route → `router.post('/upload', handler)` in `routes/files.ts`.
4. Handler reads `req.headers['x-user-id']` → `"user_1"`.
5. `const bb = busboy({ headers: req.headers })` — busboy reads `Content-Type` to learn the boundary.
6. `req.pipe(bb)` — Node starts pumping request body bytes into busboy.
7. Busboy parses the multipart envelope, fires `bb.on('file', _fieldname, fileStream, info)` with `fieldname: "file"`, a raw-bytes `fileStream`, and `info.filename: "test.txt"`, `info.mimeType: "text/plain"`.
8. `storageKey` set to `"files/4646d2a4-.../test.txt"` (UUID via `uuidv4()`).
9. `sizeCounter` Transform created — adds `chunk.length` to `size` on each chunk, calls `cb(null, chunk)`.

10. `countedStream = fileStream.pipe(sizeCounter)` — no bytes move yet.
11. `minioClient.putObject(BUCKET_NAME, storageKey, countedStream, undefined, { 'Content-Type': 'text/plain' })` — opens `PUT http://localhost:4566/vaultbox/files/4646d2a4.../test.txt`. As the SDK reads from `countedStream`, backpressure propagates back through every stage to the TCP socket. Bytes flow: **TCP → req → busboy → fileStream → sizeCounter (size += chunk.length) → MinIO SDK → HTTP PUT body → moto**.
12. moto stores the object, returns `200` to the MinIO SDK.
13. `.then()` runs: `await prisma.file.create({ filename: "test.txt", size: 51, owner: "user_1", storageKey: "files/4646d2a4.../test.txt", mimeType: "text/plain" })` → Postgres `INSERT INTO files (...) VALUES (...) RETURNING ...`.
14. `reply(201, { id, filename, size, owner, createdAt })` → `res.status(201).json({...})`.
15. curl receives `HTTP 201` with the JSON body.

`GET /files/:id`

`□`

`curl.exe -s http://localhost:4000/files/4646d2a4-... -H "X-User-Id: user_1"`

1. Express matches the route → `router.get('/files/:id', handler)`.
2. `await prisma.file.findUnique({ where: { id: "4646d2a4..." } })` → `SELECT * FROM files WHERE id = $1 LIMIT 1`.
3. `await minioClient.getObject(BUCKET_NAME, "files/4646d2a4.../test.txt")` → `GET http://localhost:4566/vaultbox/files/4646d2a4.../test.txt` to moto → returns a `Readable` stream.
4. `res.setHeader(...)` sets `Content-Type`, `Content-Disposition`, `Content-Length` from the Postgres record.
5. `stream.pipe(res)` — bytes flow: **moto → MinIO SDK Readable → res → curl stdout**.
6. curl prints: `Hello VaultBox - Stage 0 test 09/17/2026 08:51:30`

`GET /files`

`□`

`curl.exe -s http://localhost:4000/files -H "X-User-Id: user_1"`

1. Express matches the route → `router.get('/files', handler)`.
2. `owner = req.headers['x-user-id']` → `"user_1"`.
3. `await prisma.file.findMany({ where: { owner: "user_1" }, orderBy: { createdAt: 'desc' }, select: {...} })` → `SELECT id, filename, size, "mimeType", "createdAt" FROM files WHERE owner = $1 ORDER BY "createdAt" DESC`. Note: `storageKey` is not selected, so it's never fetched or returned.
4. `res.json({ files: [...] })` → curl receives `HTTP 200`.



req (IncomingMessage — TCP bytes)
→ busboy (WritableStream — multipart parser)
→ fileStream (Readable — raw file bytes emitted by busboy)
→ sizeCounter (Transform — counts bytes, passes each chunk through)
→ [MinIO SDK consumes this as the putObject body]
→ HTTP PUT body → moto

The `sizeCounter` Transform (`routes/files.ts`, lines 39–44):



```
typescript
const sizeCounter = new Transform({
  transform(chunk: Buffer, _enc, cb) {
    size += chunk.length;
    cb(null, chunk);
  },
});
```

`cb(null, chunk)` means "done processing this chunk, emit it downstream unchanged." The `null` is the error argument — `cb(new Error('bad'))` would emit an error event; `cb(null, someOtherBuffer)` would transform the bytes (how compression/encryption transforms work).

The pipeline is **demand-driven**. The MinIO SDK reads from `countedStream` only when ready to send the next chunk to moto. That pull propagates all the way back: `countedStream` ← `sizeCounter` ← `fileStream` ← busboy's buffer ← `req` ← the TCP socket. When moto is slow, backpressure propagates back to curl, which slows its send rate. **At no point does any component accumulate all the bytes** — only the current chunk (typically 64KB–1MB) exists in the heap at any moment.

The naive alternative:



```
typescript
app.use(express.raw({ limit: '10gb' }));
router.post('/upload', async (req, res) => {
  const buffer = req.body; // entire file now in heap
  await minioClient.putObject(BUCKET_NAME, key, buffer);
});
```

10 concurrent 500MB uploads → 5GB of heap → out-of-memory crash. With the streaming pipeline, heap usage is proportional to **chunk size × concurrency**, not **file size × concurrency** — a 10GB file and a 10KB file consume the same RAM during upload.

Why this matters for a file storage service specifically: the whole product promise is "store large files reliably." A server that OOMs on a 2GB upload hasn't built a file storage service.

5. The Prisma Schema

Field	Type	What it stores	Used in Stage 0?	First used by
<code>id</code>	<code>String @id @default(uuid())</code>	Primary key, UUID generated at insert	✓ Param in <code>GET /files/:id</code>	No stage changes it, but Stage 5 adds <code>parentId</code> referencing it for version chains
<code>filename</code>	<code>String</code>	Original filename from <code>info.filename</code> in busboy's file callback	✓ <code>Content-Disposition</code> on download, list response	Stage 5 adds a version number alongside it for overwrite handling
<code>size</code>	<code>Int</code>	Byte count from the <code>sizeCounter</code> Transform	✓ Upload response, <code>Content-Length</code> on download	Stage 6 runs <code>SUM(size) WHERE owner = \$1</code> for quotas — ⚠️ see overflow note below
<code>owner</code>	<code>String</code>	Raw <code>X-User-Id</code> header value, trimmed, unvalidated	✓ <code>GET /files</code> filters <code>WHERE owner = \$1</code>	Stage 6 quotas/sharing key on this; a real auth system would replace the header with a verified JWT claim
<code>storageKey</code>	<code>String</code>	S3 object key: <code>files/<uuid>/<filename></code>	✓ Passed to <code>getObject</code> on	Stage 7 cleanup scans for keys

Field	Type	What it stores	Used in Stage 0?	First used by
			download	with no matching <code>files</code> row
<code>mimeType</code>	<code>String?</code>	MIME type from busboy's <code>info</code> object, nullable	☒ <code>Content-Type</code> on download, falls back to <code>application/octet-stream</code>	Stage 4 embeds MIME type in the pre-signed URL signature
<code>createdAt</code>	<code>DateTime</code> <code>@default(now())</code>	Postgres server timestamp at insert <code>(DEFAULT NOW())</code> — never set by app code	☒ Upload response, orders <code>GET /files</code>	Stage 5 uses it to determine the latest version within a filename group

⚠️ `Int` **overflow risk:** Prisma's `Int` maps to a 32-bit signed Postgres `INTEGER`, max `2,147,483,647` bytes (~2.1GB). This isn't tied to a stage deadline — any single file over that size, uploaded today, silently wraps to a garbage negative number. Needs a `BigInt` migration before real large-file testing.

6. The Object Store Swap

The original plan was `minio/minio` from Docker Hub. It was ruled out along with every other MinIO distribution method:

Method tried

Result

`dl.min.io` binary
download

`410 Gone` — MinIO deprecated its public binary CDN; the Dockerfile's `curl` at build time failed

`docker pull minio/minio`

Auth denied — MinIO moved the image to a restricted repo requiring Docker Hub login the corporate account didn't have access to

`docker pull`
`quay.io/minio/minio`

TLS handshake failure — corporate proxy SSL inspection replaces certs; Akamai (serving quay.io) rejects the substitution at the CDN layer, before Docker even sees the registry API

`docker pull`
`localstack/localstack`

Pulls fine, but **exits code 55** — Community Edition now requires a paid Pro license for S3

`docker pull`
`motoserver/moto`

✅ **Works** — no auth, no license, S3 API functional

What changed in the code:

Two config values in `apps/api/src/lib/minio.ts`:

- **Port:** `9000` (MinIO's default) → `4566` (host port mapped to moto's container port `5000`)
- **pathStyle: true** added — without it, the SDK builds virtual-hosted URLs (`http://vaultbox.localhost:4566/<key>`), which requires DNS to resolve the `vaultbox.` subdomain on localhost — it can't. `pathStyle: true` switches to path-style URLs (`http://localhost:4566/vaultbox/<key>`), which any HTTP server handles.

`.env`/`.env.example` variable names changed from `MINIO_*` to `S3_*`; credentials became `"test"` (moto accepts any non-empty string).

Nothing in `routes/files.ts` changed. `putObject`, `getObject`, `bucketExists`, `makeBucket` all work identically against moto.

Why the same SDK works against any S3-compatible server: AWS published the S3 HTTP wire protocol as a de-facto open standard around 2006. Every object store — MinIO, moto, LocalStack, Cloudflare R2, Backblaze B2 — implements the same HTTP verbs with the same XML response format (`PUT`/`GET`/`HEAD`/`DELETE` on `/<bucket>/<key>`). The MinIO SDK just formats requests this way and parses the XML responses — it doesn't know or care what's on the other end. Two config values are sufficient to retarget it from one implementation to any other.

To switch to real AWS S3 in production: `endPoint: 's3.amazonaws.com'`, `port: 443`, `useSSL: true`, `pathStyle: false`. `routes/files.ts` stays unchanged.

7. Errors Encountered

#	Error	Cause	Fix
1	<code>dl.min.io</code> → <code>410 Gone</code>	MinIO shut down its public binary CDN; <code>minio.Dockerfile</code> 's build-time <code>curl</code> targeted a dead URL	Abandoned binary download; switched to a pre-built image
2	<code>docker pull minio/minio</code> → auth denied	MinIO restricted the Docker Hub image to authenticated accounts	Tried the quay.io mirror
3	<code>quay.io/minio/minio</code> → TLS cert verify failure	Corporate proxy SSL inspection breaks Akamai's cert pinning; fails at the CDN layer regardless of insecure-registry settings	Ruled out quay.io; tried LocalStack
4	LocalStack exit code 55	Community Edition now requires a paid Pro license for S3	Switched to <code>motoserver/moto</code> , fully open source
5	PowerShell <code>curl -F/-X</code> param errors	<code>curl</code> is aliased to <code>Invoke-WebRequest</code> in PowerShell, which uses PowerShell parameter syntax, not curl's UNIX flags	Call <code>curl.exe</code> explicitly — Windows 10+ ships the real binary at <code>C:\Windows\System32\curl.exe</code>
6	<code>moto</code> healthcheck stuck on <code>health: starting</code>	Healthcheck sent <code>GET</code> to <code>/moto-api/reset</code> ; <code>moto</code> only accepts <code>POST</code> there, returns <code>405</code> on <code>GET</code> , which Docker reads as failure — container was actually working fine	Replaced healthcheck with a Python one-liner sending a proper <code>POST</code> (Python is available inside the <code>moto</code> image)

8. What's Still Fake or Incomplete

The entire `POST /upload` handler (`routes/files.ts`, lines 14–82) treats the upload as one atomic HTTP request. Stage 1 breaks this into three separate requests:

- `router.post('/upload', ...)` is **replaced entirely** by three new endpoints:

- `POST /uploads/init` — client declares filename, total size, chunk size; receives back an `upload_id`
- `PUT /uploads/:id/chunks/:index` — client sends exactly one 5MB chunk
- `POST /uploads/:id/complete` — server assembles chunks into the final object
- `minioClient.putObject(BUCKET_NAME, storageKey, countedStream)` (line 48) currently writes the whole file in one S3 PUT. Stage 1 replaces this with per-chunk writes to a temp path (`temp/<upload_id>/chunk-<index>`), followed by a final assembly step (S3 multipart upload API or `composeObject`) at complete time.
- `prisma.file.create` (line 53) currently runs immediately after upload. Stage 1 defers this — the `File` row is only created at `/complete` time, not at individual chunk upload time. Stage 1 adds `Upload` and `Chunk` models to track partial state between `init` and `complete`.
- `size: Int` **overflow** — must migrate to `BigInt` (64-bit Postgres `BIGINT`) before real large-file testing; see Section 5.
- **No error recovery if Postgres fails after a successful S3 write** (lines 52–66) — bytes land in moto with no metadata row pointing to them, permanently invisible and undeletable. Stage 7 will garbage-collect these by scanning for storage keys with no matching `files` row.
- **No file size limit** — the handler accepts arbitrarily large single-request uploads. Stage 1's chunking caps each individual PUT at 5MB, making per-request timeouts and retries tractable.